Chapter 1: Introduction

What is the Internet?

The Internet can be defined from two perspectives:

- **From a component perspective:** The Internet is composed of billions of interconnected computing devices (called **hosts** or **end systems**), connected by **communication links** (such as optical fibers, copper cables, radios). Data is forwarded through **packet switches (routers and switches)**. It is a "**network of networks**" interconnected by various Internet Service Providers (ISPs).
- **From a service perspective:** The Internet is an infrastructure that provides services to applications (email, web, streaming, etc.). It provides a **Socket Interface** (Socket API) for distributed applications.

What is a Protocol?

A protocol defines the **format** and **order** of messages exchanged between network entities, as well as the **actions** taken on the transmission and receipt of a message. All communication in the Internet is governed by protocols (e.g., HTTP, TCP, IP, WiFi, Ethernet). Internet standards are primarily developed by the **Internet Engineering Task Force (IETF)** and published as **RFC (Request for Comments)** documents.

Network Structure

3.1 Network Edge

The "edge" of the network consists of **hosts (Clients and Servers)**, often located in **data centers**.

3.2 Access Networks

The network that physically connects an end system to the first router. Main types include:

- **Residential Access:** DSL, Cable Network (HFC), FTTH
- **Institutional Access:** Ethernet, WiFi
- **Mobile Access:** Cellular Networks (4G/5G)

3.3 Physical Media

- **Guided media (Wired):** Twisted Pair, Coaxial Cable, Fiber Optic
- **Unguided media (Wireless):** Radio signals

Network Core

4.1 Packet Switching

Working Principle: Hosts break application-layer messages into **packets**. Routers use a **Store-and-Forward** mechanism: the entire packet must be received before it can be transmitted to the next link.

Key Functions:

- **Forwarding / Switching:** Local action. Router moves arriving packets from input links to correct output links based on the **Forwarding Table**.
- **Routing:** Global action. Determines the **path** taken by packets from source to destination using **Routing Algorithms**.

Queuing and Loss: If packet arrival rate exceeds output link's transmission rate, packets queue in the buffer. If queue is full, packets are **dropped**.

Pros: Efficient for bursty data, resource sharing, simple. **Cons:** Congestion, delay, packet loss.

4.2 Circuit Switching

Working Principle: Reserve dedicated end-to-end resources for "calls", **dedicated resources, not shared**. Implemented via **Frequency Division Multiplexing (FDM)** or **Time Division Multiplexing (TDM)**.

Pros: Performance guaranteed (no congestion). **Cons:** Low resource utilization.

4.3 Packet Switching vs Circuit Switching

- **Packet-switching (Internet):** Resources are **shared**, high utilization but may queue.
- **Circuit switching (Telephone):** Resources are **exclusive**, stable quality but wasteful.

4.4 Internet Structure (Network of Networks)

Structure is hierarchical: **Access ISPs** connect to **Regional ISPs**, and Regional ISPs connect to **Tier-1 ISPs**. **Peering:** ISPs at the same level exchange traffic. **IXP (Internet Exchange Point):** Physical facility where ISPs interconnect. **CDN (Content Provider Networks):** Companies deploy data centers close to users to provide faster service.

Network Performance Metrics

5.1 Delay

Total nodal delay: $d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$

- **Processing Delay (d_{proc}):** Checking packet headers, error checking (typically μs).
- **Queuing Delay (d_{queue}):** Time waiting in output link buffer. This is the **only variable** delay.
- **Transmission Delay (d_{trans}):** Time to "push" all bits onto link. Formula: **L / R** (L=packet length, R=bandwidth).
- **Propagation Delay (d_{prop}):** Time for a bit to propagate. Formula: **d / s** (d=link length, s=propagation speed).

5.2 Packet Loss

Router output queue capacity is finite. When a packet arrives at a full queue, it will be **dropped**.

5.3 Throughput

Rate at which bits are transferred (unit: bps). End-to-end throughput = $min\{R_s, R_c\}$ (bottleneck link).

Protocol Layers and Service Models

6.1 Why Layering?

Manage Complexity and enable **Modularity** (each layer updated independently).

6.2 Internet Protocol Stack (5-Layer Model)

- **Application:** Network applications (HTTP, SMTP, DNS), unit: **message**.
- **Transport:** Process-to-process transfer (TCP, UDP), unit: **segment**.
- **Network:** Routing datagrams (IP), unit: **datagram**.
- **Link:** Adjacent node transfer (Ethernet, WiFi), unit: **frame**.
- **Physical:** Raw bits, unit: **bit**.

6.3 Encapsulation

Each layer adds its **Header** to data from upper layer: Segment = $[H_t|M]$, Datagram = $[H_n|H_t|M]$, Frame = $[H_l|H_n|H_t|M]$.

6.4 TCP/IP Model

All applications use **IP** as core network protocol. IP can run over various physical networks (Ethernet, WiFi, 4G/5G).

Network Security

Common Attacks: Packet Sniffing, IP Spoofing, Denial of Service (DoS).

Defense Measures: Authentication, Encryption, Integrity Check, Firewalls.

Brief History of the Internet

- **1961-1972:** Early packet switching, ARPANET (1969).
- **1972-1980:** Internetworking concepts, Ethernet (1976).
- **1980-1990:** TCP/IP deployment (1983), DNS (1983).
- **1990-2000s:** WWW, HTML/HTTP (Berners-Lee).
- **2005-Present:** Mobile Internet, 4G/5G, CDNs.

9.1 PAN, LAN, MAN, WAN

PAN covers a few meters, e.g., a phone connecting to Bluetooth headphones or a smartwatch. LAN serves homes or offices, such as a home Wi-Fi or an office Ethernet linking PCs and printers. MAN spans a city or large campus, e.g., city cable TV networks or university networks across campuses. WAN covers countries or continents, e.g., the Internet, multinational corporate networks, and 4G/5G mobile networks.

9.2 IANA

The Internet Assigned Numbers Authority coordinates key Internet resources: DNS root zone management, allocation of IP addresses and AS numbers to regional registries, and management of protocol parameters like port numbers. It operates under ICANN.

9.3 Satellite services in China

Examples include BeiDou (navigation and short messaging), Tiantong-1 (satellite mobile voice/data), ChinaSat (broadcast and VSAT), and developing LEO constellations like GuoWang and Qianfan.

9.4 Content providers & IXPs

Major providers (Google, Netflix, Tencent) use CDNs and private backbones. IXPs are traffic-exchange hubs earning revenue from port, membership, and colocation fees; major IXPs include DE-CIX, AMS-IX, LINX, HKIX, and Equinix IX.

Chapter 2: Application Layer

Application Layer Principles

1.1 Network Application Architectures

- **Client-Server Architecture:**
  - **Server:** Always-on host with permanent IP address.
  - **Client:** Actively contacts server, may be intermittently connected, may have dynamic IP addresses.
  - Examples: Web (HTTP), email (SMTP).
- **P2P Architecture:**
  - No always-on server. Direct communication between peers.
  - **Advantages:** Self-scalability (new peers bring new capacity).
  - **Disadvantages:** Complex management (peers are intermittently connected, IPs may change).
  - Examples: BitTorrent.

1.2 Process Communication and Sockets

- Processes communicate through **Socket** API. Socket = IP address + Port number.
- IP address identifies the host, port number identifies the process on that host.

1.3 Application Layer Protocols

- Defines message types, syntax, semantics, and rules for communication.
- **Open Protocols:** Defined in RFCs (e.g., HTTP, SMTP).
- **Proprietary Protocols:** Owned by specific companies (e.g., Skype, Zoom).

1.4 Transport Layer Services

Applications need: reliable data transfer, throughput, timing, security.

1.5 Transport Protocols

- **TCP:** Reliable, connection-oriented, provides flow/congestion control. No timing/throughput/security guarantees.
- **UDP:** Unreliable, connectionless, low overhead. Application layer can implement reliability as needed.
- **TLS:** Provides encryption, data integrity, endpoint authentication for TCP (HTTPS).

1.6 Common Applications

Web (HTTP), File transfer (FTP), P2P, Email (SMTP), DNS.

Web and HTTP

- HTTP is an application layer protocol based on TCP, using C/S architecture.
- HTTP is **stateless:** Server does not save information about past client requests.

2.1 HTTP Connection Types

- **Non-Persistent HTTP (HTTP/1.0):** Each request opens a new TCP connection. High overhead: 2 RTTs per object.
- **Persistent HTTP (HTTP/1.1):** Multiple objects on a single TCP connection. Significantly reduces latency.

2.2 HTTP Response Status Codes

200 OK, 301 Moved, 400 Bad Request, 404 Not Found, 505 Version Not Supported.

2.3 Cookies

Since HTTP is stateless, Cookies track user sessions:

- Server sends ID via `Set-cookie:` header.
- Browser stores ID in local cookie file.
- Browser includes ID via `cookie:` header in subsequent requests.
- Server uses ID to identify user and maintain state.

2.4 Web Caching

- **Goal:** Satisfy requests without accessing origin server.
- **Cache hit:** Returns object immediately.
- **Cache miss:** Requests from origin server, stores copy, returns to client.
- **Conditional GET:** Uses `If-modified-since:<date>` header. Server returns `304 Not Modified` if unchanged (saves bandwidth).

2.5 HTTP/2 and HTTP/3

- **HTTP/1.1:** FCFS scheduling, head-of-line blocking (large objects block small ones).
- **HTTP/2 (2015):** Multiplexing on single TCP connection, frame/stream interleaving, priority management. Still affected by TCP-level HOL blocking.
- **HTTP/3 (2019):** Runs on QUIC (UDP), truly solves HOL blocking, built-in TLS, faster connection establishment.

Email (SMTP, IMAP)

- **Components:**
  - **User Agents:** Email readers (Outlook, mobile apps).
  - **Mail Servers:** Store mailboxes and message queues.
  - **SMTP:** Push protocol between mail servers.
- **SMTP:** Uses TCP port 25, persistent connections, ASCII commands (`HELO` , `MAIL FROM:` , `RCPT TO:` , `DATA`).
- **IMAP:** Keeps mail on server, allows server-side management (pull).
- **Why TCP?** Requires reliable data transmission, no data loss allowed.

DNS (Domain Name System)

- **Function:** Distributed, hierarchical database resolving hostnames to IP addresses.
- **Services:** Hostname-to-IP translation, host aliasing (CNAME), mail server aliasing (MX), load distribution.
- **Why distributed?** Single centralized DNS cannot scale.

4.1 DNS Hierarchy

- **Root DNS Servers:** Top level (13 logical servers, hundreds of replicas).
- **TLD Servers:** Generic (.com, .org) and country (.cn, .uk) domains.
- **Authoritative DNS Servers:** Organizations' official DNS servers.
- **Local DNS Servers:** ISP-provided, acts as proxy forwarding queries.

4.2 DNS Resolution

- **Iterative Query:** Local DNS queries root → TLD → authoritative, each returns next server address.
- **Recursive Query:** Local DNS handles all steps (root servers usually only accept iterative).

4.3 DNS Caching

All DNS servers cache mappings. Entries expire per TTL (Time-to-Live) value.

4.4 DNS Records

Format: `(Name, Value, Type, TTL)`

- **Type=A:** Hostname → IPv4 address.
- **Type=NS:** Domain → authoritative DNS server hostname.
- **Type=CNAME:** Alias → canonical hostname.
- **Type=MX:** Domain → mail server hostname.

4.5 DNS Transport

- **UDP** (port 53) is primary (low overhead, fast).
- **TCP** used for: responses >512 bytes (DNSSEC), zone transfers.

P2P File Distribution (BitTorrent)

- **P2P vs C/S:** C/S distribution time grows linearly with N. P2P has self-scalability (peers are also uploaders).
- **BitTorrent:**
  - Files divided into 256KB chunks.
  - **Torrent:** Group of peers exchanging same file chunks.
  - **Tracker:** Server tracking peers in torrent.
  - **Mechanism:**
    - New peer gets peer list from Tracker.
    - **Rarest First:** Prioritize requesting rarest chunks.
    - **Tit-for-Tat:** Prioritize sending to peers uploading fastest.
    - **Optimistic Unchoke:** Periodically randomly unchoke a peer to find new partners.

Video Streaming and CDN

6.1 Video Streaming

- **Main consumer of network traffic:** ~80% of residential ISP traffic (Netflix, YouTube, Amazon Prime).
- **Challenges:** Scale (1 billion users), heterogeneity (device/bandwidth variations).
- **Video coding:** Compress using spatial/temporal redundancy.
- **Streaming:** Client plays first part while server sends latter part. Requires client-side buffering to handle network jitter.

6.2 DASH

- **DASH (Dynamic Adaptive Streaming over HTTP):**
  - Server: Cuts video into chunks, prepares multiple quality versions, generates manifest file.
  - Client: Downloads manifest, adaptively selects quality based on network conditions.
  - Good network → high bitrate, poor network → low bitrate.

6.3 CDN

- **Problem:** Single server cannot scale globally.
- **Solution:** Geographically distributed CDN nodes store video copies.
- **Workflow:** DNS redirects users to nearest CDN node.
- **Placement methods:**
  - **Enter Deep:** Many small servers at ISP edges (lower latency).
  - **Bring Home:** Few large data centers (easier management).

6.4 DASH, CDN, and DNS Collaboration

- User clicks video link.
- DNS judges location, redirects to nearest CDN node.
- DASH downloads manifest, requests chunks from CDN based on network speed.
- CDN transmits chunk data to user.

Socket Programming

Socket is the API for building C/S network applications.

7.1 UDP Socket

- **Connectionless:** No handshake, sender attaches destination IP/port (`sendto()`), receiver extracts sender address (`recvfrom()`).
- **Unreliable:** Data may be lost or received out-of-order.

UDP Client:

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_DGRAM)
message = input('Input lowercase sentence: ')
clientSocket.sendto(message.encode(), (serverName, serverPort))
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

UDP Server:

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(), clientAddress)
```

7.2 TCP Socket

- **Connection-oriented:** Requires connection establishment (three-way handshake).
- **Server:** socket → bind → listen → accept (creates connection socket) → send/recv → close.
- **Client:** socket → connect → send/recv → close.

TCP Client:

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence: ')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print('From Server:', modifiedSentence.decode())
clientSocket.close()
```

TCP Server:

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
while True:
    connectionSocket, clientAddr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()
```

7.3 UDP vs TCP Socket

- **UDP server:** Needs 1 socket (connectionless, all packets arrive at same socket).
- **TCP server:** Needs 2+ sockets (1 listening socket + 1 connection socket per client = 1+n for n parallel connections).
- **TCP:** Server must run first (needs to listen before client connects).
- **UDP:** Client can run first (no handshake required, though early packets may be lost).

Key Differences Summary

8.1 HTTP versions: HTTP/1.0 opens a new TCP connection per request, causing high overhead. HTTP/1.1 adds persistent connections, pipelining, and chunked transfer. HTTP/2 uses binary framing, multiplexing multiple streams on one connection, header compression, and prioritization. HTTP/3 runs over QUIC (UDP), with built-in TLS, faster handshakes, and better performance on unstable networks.

8.2 SMTP: SMTP normally uses intermediate (relay) mail servers to forward mail across domains, enabling routing, spam filtering, and message queuing.

8.3 DNS query behavior: Local DNS resolvers use recursive queries for clients. Root, TLD, and authoritative servers respond iteratively, pointing to the next server.

8.4 DNS transport: DNS mainly uses UDP port 53; TCP is used for large responses (e.g., DNSSEC) and zone transfers.

8.5 CDN placement: Deep ISP deployment places caches close to users (low latency, e.g., Akamai). Centralized IXP deployment uses large exchanges (lower cost, e.g., Cloudflare). Deep deployment historically has a larger market share.

8.6 DASH: DASH splits video into short segments at multiple bitrates; clients adaptively select quality based on network conditions.

8.7 TCP sockets: Server: socket–bind–listen–accept–send/recv–close. Client: socket–connect–send/recv–close. TCP ensures reliable, ordered delivery.

Chapter 3: Transport Layer

Overview of Transport Layer and Services

- **Primary Function:** To provide **logical communication** between **application processes** running on different hosts.
- **Relationship with Network Layer:**
  - Network Layer: Provides logical communication between **hosts**.
  - Transport Layer: Provides logical communication between **processes**.
- **Sender Side:** Receives messages from application layer, creates segments with headers, passes to network layer.
- **Receiver Side:** Receives segments from network layer, checks headers, reassembles messages, delivers to application layer.

Multiplexing & Demultiplexing

2.1 Multiplexing

Multiplexing is the process on the **source host** where data from different application processes is collected and encapsulated into transport layer segments.

2.2 Demultiplexing

- **UDP Demultiplexing:** Identified by a 2-tuple **(destination IP, destination port)**. All UDP datagrams with the same destination IP/port are directed to the same socket, regardless of source.
- **TCP Demultiplexing:** Identified by a 4-tuple **(source IP, source port, destination IP, destination port)**. Segments with different source IP/port are directed to different sockets.

Summary: Multiplexing adds port numbers when sending; demultiplexing uses port numbers to deliver to correct sockets when receiving.

Connectionless Transport: UDP

- **Characteristics:**
  - "Best effort": May be lost or delivered out of order.
  - Connectionless: No handshake, no connection state, low latency.
  - Small header overhead (8 bytes).
  - No congestion control.
  - Message-oriented, lightweight.
- **Uses:** DNS, SNMP, streaming multimedia, HTTP/3.
- **Segment Structure:** Source Port(16bit), Destination Port(16bit), Length, Checksum, Data.
- **Checksum:** Calculated using one's complement arithmetic. Sender performs 16-bit addition with wraparound carry, then bitwise negation. Receiver verifies by adding all words; result should be all 1s if no error.
- **Why use UDP?:** No connection establishment required (avoids increased round-trip latency); Simple; Small header size; No congestion control: UDP can propagate at any speed, Can function normally in congested environments.

Principles of Reliable Data Transfer (rdt)

Evolutionary model:

- **rdt 1.0:** Perfectly reliable channel (no loss, no errors).
- **rdt 2.0:** Channel may have bit errors. Introduces **ACK/NAK and retransmission (ARQ).**
- **rdt 2.1:** ACK/NAK may be corrupted. Introduces **sequence numbers** (0 and 1).
  - The introduction of sequence numbers is aimed at addressing the issue of duplicate packets caused by retransmissions, enabling the receiver to identify duplicate data and prevent redundant delivery (while also supporting in-order delivery).
- **rdt 2.2:** Removed NAK, uses **duplicate ACK** instead.
- **rdt 3.0:** Channel may have packet loss. Introduces **timer** for timeout and retransmission.

4.1 Pipelined Protocols

To improve link utilization, sender can send multiple packets before receiving ACK.

- **Go-Back-N (GBN):**
  - Sender has window of size N.
  - **Cumulative ACK:** ACK(n) indicates packet n and all before it are received.
  - Receiver discards out-of-order packets. Sender retransmits all unacknowledged packets after timeout.
- **Selective Repeat (SR):**
  - Receiver individually acknowledges each correctly received packet.
  - Receiver buffers out-of-order packets.
  - Sender only retransmits the lost packet (each packet has independent timer).

Window Size Constraint: Window Size $\leq \frac{1}{2} \times$ Sequence Number Space (to avoid ambiguity between new and retransmitted packets).

4.2 Example

Use the **Go-Back-N (GBN)** animation on the companion website.

Answers:

- The sender transmits five packets (0–4). Packet 0 is lost.
- `The receiver discards packets 1–4 because packet 0 is missing and repeatedly sends ACK for packet 0. The sender receives no ACK for packet 0, times out, and retransmits packets 0–4. The receiver then accepts them in order.`
- Packet 0 arrives correctly, but ACK 0 is lost.
- `The receiver successfully receives packets 1–4 and sends cumulative ACKs. The sender does not receive ACK 0, times out, and retransmits packets 0–4. The receiver discards the duplicate packets and sends ACKs again.`
- When six packets are sent and the window size is 5, only the first five packets can be sent immediately.
- `The sixth packet must wait until an earlier packet is acknowledged and the window slides forward.`
- **Selective Repeat vs. Go-Back-N:**
  - Selective Repeat buffers out-of-order packets, while Go-Back-N discards them.
  - Selective Repeat sends individual ACKs; Go-Back-N uses cumulative ACKs.
  - Selective Repeat retransmits only lost packets; Go-Back-N retransmits all packets after the lost one.

Connection-Oriented Transport: TCP

5.1 Overview and Segment Structure

- **Characteristics:** Point-to-point, reliable, ordered byte stream, full-duplex, connection-oriented, flow control.
- **Sequence Number (Seq):** The number of the first byte of data in the segment.
- **Acknowledgement Number (Ack):** The sequence number of the **next** byte expected (cumulative acknowledgement). Formula: ACK = Received Seq + Data Length.
- **RTT Estimation:**
  - `SampleRTT` : Measured time from transmission to ACK receipt.
  - `EstimatedRTT = (1−α) × EstimatedRTT + α × SampleRTT`
  - `TimeoutInterval = EstimatedRTT + 4 × DevRTT`
- **Receive Window (rwnd):** Flow control - number of bytes receiver is willing to accept.

5.2 Handling of Out-of-Order Segments

TCP receiver buffers out-of-order bytes and waits for missing bytes to fill gaps (more efficient than discarding).

5.3 TCP Receiver: ACK Generation Rules

- **In-order segment, all data ACKed:** Delayed ACK (wait up to 500ms).
- **In-order segment, another pending ACK:** Immediately send cumulative ACK.
- **Out-of-order segment:** Immediately send duplicate ACK to trigger fast retransmit.

5.4 Reliable Data Transfer

Retransmission Triggers:

- **Timeout:** Retransmits segment with smallest unacknowledged sequence number, doubles timeout interval.
- **Fast Retransmit:** If **3 duplicate ACKs** received, segment considered lost and retransmitted immediately.

5.5 Flow Control

- **Purpose:** Prevent sender from overwhelming receiver's buffer.
- **Mechanism:** Receiver announces remaining buffer size in `rwnd` field.
  - `rwnd = RcvBuffer − (LastByteRcvd − LastByteRead)`
  - Sender limits: `LastByteSent − LastByteAcked ≤ rwnd`
  - If `rwnd = 0` , sender stops and sends window probe.

5.6 Connection Management

- **Three-way Handshake:**
  - Client sends `SYN` (seq=x).
  - Server replies `SYNACK` (seq=y, ack=x+1).
  - Client sends `ACK` (ack=y+1).
- **Four-way Wave:**
  - Client sends `FIN` → Server replies `ACK` .
  - Server sends `FIN` → Client replies `ACK` .
  - Client enters `TIME_WAIT` state (2MSL).

Congestion Control

- **Causes:** Throughput cannot exceed link capacity; delay increases rapidly near capacity; packet loss reduces effective throughput.
- **Purpose:** Prevent too much data from overloading network core (routers).
- **Signs:** Packet loss, long delays.
- **AIMD:** Additive Increase (cwnd +1 MSS per RTT) + Multiplicative Decrease (cwnd halved on loss).
- **TCP Congestion Control (Reno):**
  - **Slow Start:** cwnd starts at 1 MSS, doubles per ACK (exponential) until `ssthresh` .
  - **Congestion Avoidance:** When cwnd ≥ `ssthresh` , increases linearly (+1 MSS per RTT).
  - **Fast Recovery:** After 3 duplicate ACKs, `ssthresh = cwnd/2` , `cwnd = ssthresh + 3` , enter congestion avoidance.
  - **Timeout:** `ssthresh = cwnd/2` , `cwnd = 1 MSS` , re-enter slow start.

6.1 Queue Management (AQM)

- **Tail Drop:** Discards all packets when queue full → causes **Global Synchronization**.
- **RED (Random Early Detection):** Randomly discards packets before queue full based on average queue length.
- **WRED:** Different drop thresholds for different traffic priorities.

6.2 TCP Variants

TCP CUBIC: Uses cubic function for window adjustment in high-bandwidth-delay-product networks. Default for Linux.

6.3 Supporting Mechanisms and Fairness

ECN (Explicit Congestion Notification): Network-assisted congestion control using IP header bits to signal congestion without packet loss.

TCP Fairness:

- **Goal:** Each of K TCP sessions should get R/K bandwidth on bottleneck link.
- **AIMD mechanism** can achieve fairness under ideal conditions (same RTT, fixed sessions).
- **Limitations:** UDP traffic doesn't respond to congestion; multiple TCP connections per application; different RTTs affect fairness.

Evolution of Transport Layer: QUIC and HTTP/3

- **Background:** TCP + TLS handshake latency high; Head-of-Line (HOL) blocking problem.
- **QUIC:** Application-layer protocol based on **UDP**.
  - Implements reliable transmission, congestion control, encryption.
  - **0-RTT / 1-RTT connection establishment.**
  - **Multi-stream Multiplexing:** Eliminates HOL blocking (loss in one stream doesn't block others).

Key Concepts Summary

8.1 TCP vs GBN vs SR

TCP behavior is closer to Selective Repeat (SR): receiver buffers out-of-order segments, uses fast retransmit and SACK. However, TCP uses cumulative ACKs, so it's not pure SR.

8.2 Flow Control vs Congestion Control

- **Flow Control:** Prevents sender from overwhelming **receiver's buffer** (end-to-end, via rwnd).
- **Congestion Control:** Prevents too much data in the **network** (global, via cwnd).

8.3 Global Synchronization

When multiple TCP connections share bottleneck link, they detect congestion simultaneously and reduce rates together, causing periodic fluctuations. Caused by tail-drop policy.

8.4 TCP Congestion Control Process

Four phases: **Slow Start** (exponential growth), **Congestion Avoidance** (linear increase), **Fast Retransmit** (3 duplicate ACKs), **Fast Recovery** (adjust cwnd, skip slow start). On timeout, reset to slow start.

8.5 TCP Fairness Issues

Fairness depends on similar RTTs and same algorithm. Real-world issues: different RTTs, multiple connections per app, UDP traffic.

8.6 Why UDP?

No connection establishment, low latency, low overhead, no reliability/flow/congestion control, greater application control. Suitable for real-time applications tolerating packet loss.

8.7 Why TCP for Voice/Video?

TCP more easily traverses firewalls/NATs, widely supported, many apps use HTTP/HTTPS over TCP. Not related to congestion control.

8.8 Reliable Transfer over UDP

Applications can implement reliability at application layer: sequence numbers, acknowledgements, timeout retransmissions.

Principles and Application Scenarios of TCP and UDP

- TCP
  - Principles: Connection-oriented, reliable, and in-order transmission. Establishes a connection through a three-way handshake, and ensures data integrity and correctness through acknowledgment, retransmission, flow control, and congestion control.
  - Application Scenarios: Services with high requirements for reliability and integrity, such as HTTP1.1/2, FTP, SMTP, SSH, etc.
- UDP
  - Principles: Connectionless, best-effort, does not guarantee reliability or order, directly sends datagrams, has low overhead and low latency.
  - Application Scenarios: Services with high real-time requirements and tolerance for minor packet loss, such as audio and video, online games, DNS, QUIC/HTTP/3, DHCP, etc.

Similarities and Differences Between TCP and UDP (Simplified Comparison)

- Similarities
  - Both are transport layer protocols
  - Both use port numbers
  - Both have error checking mechanisms
- Differences
  - Connection Method: TCP is connection-oriented; UDP is connectionless
  - Reliability: TCP is reliable; UDP is unreliable
  - Transmission Method: TCP is byte-stream oriented; UDP is datagram-oriented
  - Performance Characteristics: TCP is stable but has higher latency; UDP is fast but may drop packets
  - Control Mechanisms: TCP has flow and congestion control; UDP does not have these mechanisms at the protocol layer

Chapter 4 Network Layer: Data Plane

Network Layer Overview: Data Plane and Control Plane

- **Data Plane:** Local, router-based function. Main function is **Forwarding:** determining how datagrams arriving at router input ports move to appropriate output ports.
- **Control Plane:** Network-wide logic. Main function is **Routing:** determining the end-to-end path from source to destination for datagrams (routing algorithms).
  - Two implementation methods:
    - Traditional method: routing algorithms are implemented in each router.
    - SDN (Software-Defined Networking): remote controllers compute and install forwarding tables in routers.

Difference: Forwarding is the process by which routers send arriving packets from input interfaces to correct output interfaces according to forwarding tables, while routing is the process of computing end-to-end paths network-wide through routing algorithms and routing protocols and generating forwarding tables.

Service Model

The Internet provides a **Best Effort** service model, which does not guarantee bandwidth, packet loss rate, order, or delay. The simplicity enables widespread deployment, and sufficient bandwidth plus distributed services (CDN, data centers) make performance "good enough for most users."

Router Internal Structure

Routers are mainly composed of input ports, switching fabric, output ports, and routing processor.

3.1 Two Main Components

- **Control Plane:** Routing processor, implemented by software, executes routing protocols, operates at millisecond level.
- **Data Plane:** Implemented by hardware, includes input/output ports and switching fabric, operates at nanosecond level.

3.2 Router Structure Components

3.2.1 Routing Processor

- Executes control plane functions
- Routing table stored here, generates forwarding tables

3.2.2 Shadow Copy

- Forwarding table copied to each line card to avoid centralized bottlenecks
- Each input port has local copy for parallel, nanosecond-level forwarding

3.2.3 Input Ports

- Execute physical and data link layer functions

- Store forwarding tables mapping destination address prefixes to next hop/output interfaces
- **Forwarding methods**:
  - Destination-based forwarding: forward only based on destination IP address (longest prefix match)
  - Generalized forwarding: forward based on any set of header field values
- **HOL Blocking (Head-of-Line Blocking)**: Datagrams at front of queue blocked, blocking subsequent packets. Solution: Virtual Output Queues (VOQ) - each input port maintains separate queue for each output port.

3.2.4 Switching Fabrics

Three main types:

- **Memory-based switching**: Limited by memory bandwidth, one packet at a time
- **Bus-based switching**: All inputs share one bus, one packet at a time, bus contention exists
- **Interconnection Network switching**: Crossbar switching, foundation of modern high-performance routers. Supports parallel multi-plane architecture for scaling.

3.2.5 Output Ports

- **Buffering**: When datagrams arrive faster than link transmission rate
  - Buffer size: $\text{Buffer Size} = \frac{RTT \times C}{\sqrt{N}}$ (N: TCP connections, RTT: round-trip time, C: link bandwidth)
  - Excessive buffering causes high delay
- **Packet Scheduling**:
  - FIFO: First come first served
  - Priority: High priority queues sent first
  - Round Robin: Cyclically serve queues
  - Weighted Fair Queue (WFQ): Allocate bandwidth by weight. Bandwidth for queue i: $R_i = R \times \frac{w_i}{\sum w_j}$

IP Protocol (IPv4)

4.1 Datagram Format

Key fields:

- **ver**: IP version (IPv4 is 4)
- **head. len**: Header length (usually 20 bytes)
- **type of service**: QoS, Differentiated Services, ECN
- **length**: Total datagram length
- **identifier/flags/fragment offset**: Fragmentation and reassembly fields
- **TTL**: Time to live, decremented by 1 each hop, discarded at 0
- **upper layer (protocol)**: Upper layer protocol type (TCP=6, UDP=17)
- **header checksum**: Checks header errors (only covers IP header)
- **source/destination IP address**: 32-bit addresses
- **Fixed header length**: 20 bytes

4.2 Fragmentation and Reassembly

- Network links have Maximum Transmission Unit (MTU)
- Large IP datagrams fragmented in network, reassembled at destination
- When IP datagram exceeds link MTU, fragmented by routers, reassembled only at destination host

4.3 IP Addressing

- **IP Address**: 32-bit identifier for host or router interfaces
- **Subnet**: Isolated network where devices can physically reach each other without routers
- **CIDR**: Format `a.b.c.d/x`, where `x` is subnet mask length, supports arbitrary length prefixes, helps route aggregation

4.4 DHCP

DHCP (Dynamic Host Configuration Protocol): Plug-and-play protocol allowing hosts to dynamically obtain IP addresses:

- Process: Discover → Offer → Request → ACK
- Based on UDP
- Returns: assigned IP address, first-hop router IP (default gateway), DNS server address, network mask

4.5 NAT

- Allows multiple internal hosts to share one public IP, distinguishing connections through port numbers
- Routers replace source IP addresses and port numbers, maintain NAT translation tables
- Controversy: Violates end-to-end principle, but effective for IPv4 address shortage

IPv6

5.1 Improvements

- **Motivation**: IPv4 address exhaustion
- **128-bit address space**
- **Fixed 40-byte basic header**, removed checksum and fragmentation fields (routers do not fragment)
- **Introduced Flow Label**

5.2 Comparison with IPv4

IPv6 removes checksum, fragmentation, options from fixed header (moved to extension headers), adds Traffic Class and Flow Label.

5.3 Transition Mechanism

Tunneling: Encapsulates IPv6 datagrams in IPv4 datagrams, enabling IPv6 packets to traverse IPv4-only networks.

Generalized Forwarding and SDN

6.1 Generalized Forwarding

- Abstracts packet processing as **match + action**
- **Traditional**: Destination-based forwarding using longest prefix match
- **Generalized**: Match on multiple header fields (source/destination IP, protocol, ports, VLAN ID, etc.), actions include forward, drop, modify, copy, record

6.2 SDN

- **SDN (Software-Defined Networking)**: Separates control plane from data plane, centralized control through flow tables
- **OpenFlow**: Southbound Interface protocol for SDN controllers and network devices
  - **Match**: Link layer (MAC), network layer (IP), transport layer (Port)
  - **Actions**: Forward, drop, modify fields, encapsulate and forward to controller
  - **Applications**: Routers, switches, firewalls, NAT, load balancing

Middleboxes

- **Definition**: Network devices that inspect, modify, filter, or redirect packets
- **Functions**:
  - NAT Translation: Rewrite IP addresses and port numbers
  - Security Services: Firewalls, IDS (Intrusion Detection Systems), DPI (Deep Packet Inspection)
  - Performance Enhancement: Load balancing, content caching
- **Architectural Impact**: Breaks traditional network layering

Key Questions

8.1 What is HOL blocking? Where does it occur?

HOL (Head-of-Line) blocking: Packets at head of input port queue unable to forward because target output ports are busy, blocking subsequent packets. Occurs at **input ports**. Solution: Virtual Output Queues (VOQ).

8.2 Packet loss at input ports: reasons and solutions

When packet arrival rate exceeds switching fabric or output port processing rate, packets queue in input buffers. If buffers are full, packet loss occurs. Solutions: improve switching fabric scheduling efficiency, adopt VOQ to avoid HOL blocking, perform traffic scheduling and rate control.

8.3 Packet loss at output ports: can increasing switching fabric speed prevent it?

When packets from multiple input ports forward to same output port and arrival rate exceeds link transmission rate, output buffers overflow causing packet loss. Increasing switching fabric speed reduces delays but cannot prevent loss, as output link bandwidth is still limited.

8.4 Why is IPv6 deployment slow?

IPv4 address exhaustion alleviated by NAT, reducing urgency. IPv6 not fully compatible with IPv4, migration requires transition mechanisms (dual-stack, tunneling), increasing complexity and costs. Equipment upgrades and training require significant investment.

8.5 Compare IPv4 and IPv6

IPv6 advantages: Larger address space (128-bit), simpler header design (removes checksum, fragmentation fields), improves router forwarding efficiency, native IP-layer security, better QoS and mobility support, reduced NAT dependence.

8.6 Firewall, IDS, and IPS differences

- **Firewall**: Filters and controls network traffic according to security policies, decides which traffic passes or is blocked.
- **IDS (Intrusion Detection System)**: Monitors abnormal behavior, discovers potential attacks by analyzing traffic/logs, provides alerts but does not directly block traffic.
- **IPS (Intrusion Prevention System)**: Adds active defense on top of IDS, automatically drops malicious traffic or blocks connections when attacks detected.

8.7 IPv6 tunneling through IPv4

Logically, IPv6 treats IPv4 tunnel as link-layer protocol: IPv6 packets encapsulated in IPv4 packets, tunnel provides transparent transmission service. However, from strict protocol layering perspective, IPv4 remains network layer protocol, so this is more of a functional analogy.

8.8 Interfaces and forwarding table lookups

With 3 routers between source and destination: IP datagram passes through 8 interfaces (1 source host + 6 router interfaces + 1 destination host). Forwarding tables looked up 3 times (once per router).

8.9 Switching fabric parallel forwarding

- Shared bus: No, can only transmit one packet at a time.
- Memory-based: No, can only process one packet at a time.
- Crossbar to same output: No, same output port can only receive one packet at a time.

Chapter 5 Network Layer: Control Plane Knowledge Points Summary

Overview: Two Methods of Building the Control Plane

The main functions of the network layer are divided into forwarding (data plane) and routing (control plane). The control plane is primarily responsible for computing routing forwarding tables.

1.1 Per-router control

- **Traditional approach**: Each router runs routing algorithms, exchanges control plane information, and independently computes forwarding tables.
- Both control plane and data plane are implemented in the same router.
- Representative protocols: OSPF, BGP.

1.2 Logically centralized control

- **SDN (Software Defined Networking)**: A remote, logically centralized controller computes and distributes forwarding tables.
- Routers only communicate with the controller and perform forwarding, without computing routes.

Routing Algorithms

Goal: Determine a "good" path from source to destination. Network abstracted as graph $G = (N, E)$.

2.1 Algorithm Classification

- **Centralized vs Decentralized**:
  - **Centralized**: All routers have complete network topology (Link State, Dijkstra).
  - **Decentralized**: Routers only know neighbor link costs (Distance Vector, Bellman-Ford).
- **Static vs Dynamic**: Static routes change slowly (manually configured); Dynamic routes respond quickly to changes.
- **Load-sensitive vs Load-insensitive**: Load-insensitive (current Internet mainly uses this, e.g., RIP, OSPF, BGP) for stability and to avoid oscillations.

2.2 Link State Routing Algorithm (LS) - Dijkstra

- **Prerequisite**: All nodes have complete network topology (achieved through link state broadcast).
- **Characteristics**: Centralized, iterative.
- **Process**: Record distance to neighbors → pick closest node → confirm shortest path → update distances → repeat.
- In Link-State protocols (OSPF, IS-IS): All routers obtain complete topology through LSA flooding, each runs Dijkstra locally, computes shortest paths.
- **Complexity**: $O(n^2)$ or $O(n \log n)$; Message complexity: $O(n^2)$.
- **Problem**: Routing oscillations may occur when link costs are related to traffic.

2.3 Distance Vector Routing Algorithm (DV) - Bellman-Ford

- **Core idea**: Each node periodically sends distance vector estimates to neighbors. When x receives new DV from neighbor, updates using B-F equation: $Dx(y) = \min\{c(x,v) + Dv(y)\}$.
- **Characteristics**: Iterative, asynchronous, distributed.
- **Key problems**:
  - **Good news travels fast**: Cost reductions spread quickly.
  - **Bad news travels slow (Count-to-infinity)**: Cost increases spread slowly, may cause routing loops.
  - Solution: Poisoned Reverse.

2.4 Comparison of LS and DV

Feature	Link State (LS)	Distance Vector (DV)
Message Complexity	High ($O(nE)$)	Lower (neighbor exchange only)
Convergence Speed	Fast ($O(n^2)$), may oscillate	Slow, routing loops possible
Robustness	Stronger (errors affect locally)	Weaker (errors propagate)

> ***Count-to-infinity***: *In DV algorithms, when a path fails, routers may repeatedly believe a reachable path exists, causing path costs to continuously increase.*

AS (Autonomous System)

An Autonomous System (AS) is a network under single administrative control, using a single routing protocol, identified by an AS number.

3.1 AS Classification

- **Multi-homed AS**: Connected to ≥ 2 ASes, for redundancy/reliability.
- **Stub AS**: Connected to only 1 external AS.
- **Transit AS**: Connected to multiple ASes, allows traffic that does not belong to itself to pass through.

3.2 Two Levels of Routing

- **Intra-AS (Intra-domain)**: Within same AS, all routers run same protocol, goal: fast, stable, shortest path.
- **Inter-AS (Inter-domain)**: Between ASes, handled by Gateway Routers, goal: **Policy first**, not necessarily shortest path.

3.3 Routing Decision Sources

- For destinations within AS: determined only by **intra-AS routing**.
- For destinations outside AS: inter-AS determines "which AS to exit through", intra-AS determines "how to reach that exit router".

IGP (Interior Gateway Protocol)

Most common intra-domain routing protocols: RIP, OSPF, IS-IS.

4.1 RIP (Routing Information Protocol)

- Based on Distance Vector (DV) algorithm, metric: **Hops**, maximum 15 hops (16 = unreachable).
- Exchange routing information via UDP every 30 seconds.
- **Timers**: Update (30s), Invalid (180s), Hold-down (180s), Garbage Collection (120s).
- Good news travels fast, bad news travels slow.

4.2 OSPF (Open Shortest Path First)

- Based on Link State (LS) algorithm (Dijkstra).
- Uses IP datagrams directly (not TCP/UDP).
- Supports multiple metrics, security (authenticated messages), multiple equal-cost paths (load balancing).
- Uses flooding to send information to all routers.
- **Hierarchical Structure**: Two levels - Backbone area (Area 0) and Local Areas. All areas must connect to backbone.
- **LSA (Link State Advertisement)**: Contains neighbor path cost information.

Inter-AS Routing: BGP

Inter-AS routing protocol handling path selection between different ASes.

5.1 BGP (Border Gateway Protocol)

- Inter-domain routing protocol.
- Routes to CIDRized prefixes, not specific addresses.
- **Types**:
  - **eBGP (External BGP)**: Obtains subnet reachability from adjacent ASes.
  - **iBGP (Internal BGP)**: Propagates external reachability to all routers within AS.
- BGP is commonly used to implement IP anycast (especially in DNS).

5.2 BGP Basics

- **BGP Session**: Between peers via TCP (port 179), exchanges path information.
- **Path Vector Protocol**: Carries complete AS-level path information (unlike DV).
- **BGP Advertisement**: A forwarding promise - "If you give me traffic to X, I will forward it".

5.3 BGP Path Attributes

- **AS-PATH**: List of ASes through which prefix advertisement passed. Used for loop detection and policy decisions.
- **NEXT-HOP**: Specific router interface IP to reach next-hop AS.
- **Policy-based routing**: Import Policy (accept/modify/reject), Export Policy (selective advertisement).

5.4 BGP Route Selection Steps

When router learns multiple paths, select in order:

- **Local Preference**: Highest (set by administrator).
- **AS-PATH**: Shortest (fewest ASes).
- **NEXT-HOP**: Nearest (Hot Potato Routing - choose exit with smallest internal cost).

5.5 iBGP vs eBGP

Attribute	iBGP	eBGP
Scope	Within same AS	Between different ASes
Route advertisement	Only to iBGP neighbors	To all eBGP neighbors
Topology	Requires full mesh	Not required
Loop Prevention	BGP split horizon	AS-PATH

Key points: iBGP split horizon rule prevents loops within AS. iBGP is not IGP. BGP + IGP enable forwarding: BGP tells exit (next-hop), IGP tells how to reach exit.

Software Defined Network (SDN) Control Plane

SDN separates and centralizes **control plane** (deciding paths) from **data plane** (forwarding packets).

6.1 Architecture Components

- **Network-control apps**: Determine control logic (routing, access control, load balancing). Can utilize global network view.
- **SDN Controller**: Maintains network state (topology, link state, host information). Logically centralized, physically distributed.
  - **Northbound API**: Communicates with applications (e.g., RESTful API).
  - **Southbound API**: Communicates with data plane devices (e.g., OpenFlow).
- **Data plane switches**: Fast hardware, perform Flow Table matching and actions.

6.2 OpenFlow Protocol

- Runs between controller and switches (TCP/TLS).
- **Message types**: Controller-to-switch (read state, configure, modify state, packet output), Asynchronous (switch-to-controller), Symmetric (heartbeat).

6.3 SDN Example

- Switch detects link failure → tells Controller via OpenFlow port-status message
- Controller updates state database → calls Dijkstra routing application
- Routing application recalculates routes → Controller distributes new flow tables to switches

ICMP (Internet Control Message Protocol)

- **Purpose**: Network layer information exchange between hosts and routers.
- **Main functions**: Error reporting (Destination unreachable, TTL expired), Echo request/reply (`ping`).
- **4 Types of ICMP Messages**:
  - **Echo Request/Reply**: Connectivity testing (`ping`).
  - **Destination Unreachable**: Cannot reach destination.
  - **Time Exceeded**: TTL expired (used in `traceroute`).
  - **Redirect**: Notify better next-hop routes.
- **Location**: Encapsulated in IP datagrams.

7.1 Traceroute Example

- Source sends UDP probe packets with increasing TTL (1, 2, 3...)
- Each router decrements TTL; when TTL=0, router returns **ICMP Time Exceeded (Type 11, Code 0)**
- When packet reaches destination, destination returns **ICMP Port Unreachable (Type 3, Code 3)**
- Source records each hop's IP and RTT to discover path

Network Management

8.1 Evolution of Management Methods

- **CLI**: Error-prone, difficult to scale.
- **SNMP**: Application layer protocol, uses UDP. **MIB** (Management Information Base) stores device data. Modes: Request/Response and Trap.
- **NETCONF/YANG**: Modern approach. YANG defines data structures, NETCONF transmits configuration. Based on XML, TLS/TCP, supports atomic transactions.

8.2 SNMP

- **Manager**: Monitors, controls, configures managed devices.
- **Agent**: Software on managed devices, maintains management information.
- **MIB**: Database describing management information.
- **GetRequest/SetRequest**: Read/modify MIB values.
- **Trap**: Device actively reports events/anomalies.

HomeWork

9.1 Compare LS and DV Routing Algorithms, and Compare Static Routing with Dynamic Routing

Link State: Each router obtains full topology through flooding, runs Dijkstra locally. Advantages: fast convergence, stable, less loop sensitivity. Disadvantages: topology database overhead, flooding overhead. Example: OSPF.

Distance Vector: Routers know distance estimates, exchange with neighbors using Bellman-Ford. Advantages: simple, local information only. Disadvantages: count-to-infinity, slow convergence, loop risk. Example: RIP.

Static routing: Manually configured, stable, no protocol overhead, but high maintenance cost, cannot adapt to topology changes.

Dynamic routing: Automatically learns/updates routes, adapts to failures, but has control overhead and complexity.

9.2 Most Common Internet Routing Protocols (RIP, OSPF, BGP) Are Mostly Load-insensitive. Is This True? How to Solve Problems?

Yes. Traditional protocols base path selection on static/semi-static metrics (RIP: hop count, OSPF: configured cost, BGP: policy), not real-time link load.

Problems: Traffic concentrates on certain paths causing congestion while other paths idle, unbalanced utilization, increased delay.

Solutions: Traffic Engineering (MPLS-TE, SR-TE), ECMP, metric-adjustable IGP schemes, BGP policy engineering, application-layer scheduling/CDN/Anycast.

9.3 Give a BGP Accident Case and Explain the Reason

2008 Pakistan Telecom YouTube incident: Operator configured route pointing YouTube prefixes to black hole, then incorrectly published more specific prefix upstream via BGP. Due to longest prefix matching, many networks chose this incorrect route, causing global YouTube access issues.

Root cause: BGP lacks mandatory source verification, combined with configuration errors and lax upstream filtering.

9.4 iBGP Split Horizon Rule: Basic Content and Necessity

Content: Routes learned from iBGP neighbor are not advertised to another iBGP neighbor. Routes learned via iBGP can only be advertised to eBGP neighbors or special nodes (Route Reflectors).

Necessity: Prevents routing loops within AS (AS-PATH doesn't increase within AS). Consequence: requires full mesh (or Route Reflectors/Confederations for scaling).

9.5 Why Does the Internet Use Both inter-AS and intra-AS Protocols?

Internet consists of many ASes with different goals, policies, and trust boundaries. **intra-AS**: Focus on efficiency, fast convergence, shortest path (OSPF, IS-IS, RIP). **inter-AS**: Focus on policy control, scalability, business relationships (BGP).

9.6 List Four Different Types of ICMP Messages

- Echo Request/Reply: Connectivity testing (`ping`)
- Destination Unreachable: Cannot reach destination
- Time Exceeded: TTL expired (used in `traceroute`)
- Redirect: Notify better next-hop routes

9.7 BGP Is Commonly Used to Implement IP Anycast (Commonly Used in DNS)

Core approach: Multiple geographically dispersed sites announce same IP prefix via BGP. Traffic directed to "closest/optimal" site according to BGP route selection.

DNS benefits: Reduced latency, improved availability, enhanced DDoS resistance, automatic failover.

Limitations: "Closest" not necessarily geographically closest, traffic drift may occur, not suitable for long connections requiring session stickiness.

9.8 Request-Response Mode vs Trap Mode: Advantages and Disadvantages

Overhead: Request/Response needs periodic polling (large overhead); Trap only sends when events occur (small overhead).

Notification time: Request/Response depends on polling period (delayed); Trap is immediate.

Robustness: Request/Response can retry (robust); Trap may be lost (less robust).

9.9 Reasons Why SNMP Designers Chose UDP Rather Than TCP

SNMP messages are small, interactions simple, reliability requirements not high. UDP more likely to deliver during network congestion/failures, more suitable for asynchronous Trap messages.

True/False Questions

10.1 OSPF Routers Only Send Link State Information to Directly Adjacent Nodes

False. OSPF uses flooding to send information to all routers in the autonomous system, not just adjacent nodes.

10.2 After a BGP Router Receives a Path, Must It Add Its Own Identifier and Send to All Neighbors?

No. BGP routers only when deciding to advertise this route to a neighbor will: add their own AS number to AS-PATH. Whether to advertise depends on routing policy and neighbor relationship (customer/peer/provider). Will not unconditionally forward all routes to all neighbors.

Chapter 6: Link Layer and LANs

Introduction & Services

1.1 Core Concepts

- **Nodes**: Hosts, routers, switches, WiFi access points.
- **Links**: Communication channels connecting adjacent nodes (wired, wireless, LAN).
- **Frames**: Data packets at the link layer, encapsulating datagrams from the network layer.
- **Responsibility**: Transport datagrams from one node to the next physically adjacent node.

1.2 Link Layer Services

- **Framing**: Encapsulate datagrams by adding headers and trailers.
- **Link Access**: Medium Access Control (MAC), handling collisions in shared media.
- **Reliable Delivery**: Implemented between adjacent nodes (commonly used in wireless networks, usually not provided in wired networks like Ethernet). Even if all links provide reliable delivery, TCP is still not redundant because link-layer reliability is hop-by-hop, while TCP provides end-to-end reliable transport.
- **Error Detection/Correction**: Handling signal attenuation and noise.

Link layer protocols provide framing, error detection, and hop-by-hop reliability to the network layer, while end-to-end reliable delivery, flow control, and congestion control are only implemented in TCP.

1.3 Implementation Location

- **NIC (Network Interface Card)**: The link layer is mainly implemented in the NIC (hardware), and partially in operating system drivers (software).

Error Detection and Correction

2.1 Error Detection

Three main techniques:

- **Parity Checking**: Single-bit parity detects single-bit errors. Two-dimensional parity can detect and correct single-bit errors.
- **Internet Checksum**: Mainly used at the transport layer (TCP/UDP), treating data as a sequence of 16-bit integers, summing and taking the complement.
- **CRC (Cyclic Redundancy Check)**: Commonly used at the link layer, based on polynomial division. Strong detection capability, widely used in Ethernet and WiFi.

Note: Error detection is not 100% reliable, but larger EDC fields can provide better detection and correction effects.

2.2 Error Correction

Forward Error Correction (FEC) and Hamming Code.

Multiple Access Protocols

Used to solve collision problems in **broadcast channels**.

3.1 Four "Desired Characteristics"

- **High efficiency when a single node exclusively occupies the channel**
- **Fairness when multiple nodes are active**
- **Decentralization (no central control)**
- **Simple protocol with small overhead**

3.2 Channel Partitioning Protocols

- **TDMA (Time Division)**: Take turns sending in time slots.
- **FDMA (Frequency Division)**: Divide channels by frequency.
- **Characteristics**: High efficiency under high load, low efficiency under low load (resource waste).

3.3 Random Access Protocols

Nodes send at full speed, retransmit after collisions occur.

- **ALOHA**:
  - Pure ALOHA: Very low efficiency (18%).
  - Slotted ALOHA: Slotted ALOHA divides time into slots and allows nodes to transmit only at slot boundaries. If multiple nodes transmit in the same slot, a collision occurs. After collision, nodes retransmit with probability p to randomize access and avoid repeated collisions, efficiency improves (37%).
    - Characteristics of slotted ALOHA: Slotted ALOHA has the characteristics of decentralization and protocol simplicity, but cannot fully utilize the channel when a single node is active, nor guarantee strict fairness.
- **CSMA (Carrier Sense Multiple Access)**: "Listen before talk" (LBT). If the channel is idle, send; if busy, defer. Collisions may still occur (due to propagation delay).
- **CSMA/CD (with Collision Detection)**: Used in **wired Ethernet**. Listen while talking, stop immediately upon detecting collision and send a jam signal.
  - Carrier Sense (CS): Before transmission, the channel is listened to; data is sent only if the channel is idle; otherwise, transmission is deferred.
  - Multiple Access (MA): Multiple hosts share a single broadcast channel and compete to transmit data.
  - Collision Detection (CD): During transmission, the channel is monitored; if a collision is detected, transmission is immediately halted, a jam signal is sent, and the transmission is retried after a backoff period.
  - **Binary Exponential Backoff Algorithm:** The more collisions occur, the larger the random range of waiting time.
- **CSMA/CA**: Used in wireless LANs (802.11), because collisions are difficult to detect in wireless, can only try to avoid them.
  - To ensure that the sender can detect the worst-case collision before the transmission is completed. If the frame is too short, the collision might only arrive after the transmission has ended, leading the sender to incorrectly assume success. Therefore, a minimum frame length is specified to ensure that the transmission time is greater than or equal to the round-trip signal propagation delay, thereby ensuring that any collision can be detected. For example, in a classic 10 Mb/s Ethernet, the minimum frame length is 64 bytes (including the header and FCS), corresponding to a "collision window" of 512 bit times, which is used to ensure that collisions can be detected.

3.4 Taking Turns Protocols

- **Polling**: Master node invites slave nodes to send (e.g., Bluetooth). Disadvantages are polling overhead and single point of failure.
- **Token Passing**: Token is passed among nodes, only the token holder can send. Efficient and fair, but complex with large control overhead.

LANs

4.1 MAC

- **MAC Address**: 48 bits, physical address, burned into NIC, globally unique, flat structure (portable).
- **Purpose**: Within the same LAN, deliver frames from one interface to another interface.
- **Important point:** It's not "hosts" that have MAC/IP, but "network interfaces" that have them. A host with multiple NICs has multiple MACs. Each interface has 1 MAC and 1 IP.
- In end-to-end communication: IP addresses remain unchanged; MAC addresses change at every hop.

4.2 ARP

- ARP's role: Within the same LAN, resolve "target IP" into "target MAC".
- Execution process:
  - **Look up table:** Host first checks its own ARP table to see if there is `target IP → MAC`
  - **If not found, broadcast:** Send ARP request to LAN broadcast address, asking: "Who has this IP?"
  - **Target responds:** The host with that IP replies with its MAC address via **unicast**
  - **Cache result:** Requesting party stores `IP → MAC` in ARP table (with expiration time)
  - **Send data:** Use the obtained MAC to encapsulate Ethernet frame and send
- ARP queries need to be sent via broadcast because the sender does not yet know the destination host's MAC address; ARP responses are sent via unicast because the responder has already learned the requester's MAC address from the query message, no need to broadcast again.

4.3 Ethernet

- Ethernet is the most mainstream wired LAN technology.
- Connectionless: No handshake process between sending and receiving network interface cards
- Unreliable: Receiving network interface cards do not send acknowledgments (ACK) or negative acknowledgments (NAK) to sending network interface cards (no ACK)

4.3.1 Ethernet Topology

- **Bus type (early):** All hosts connected to the same coaxial cable. All nodes in the same collision domain. Must use CSMA/CD.
- **Switched (mainstream today):** Center is Layer-2 switch, each host has an independent link. Each link is an independent collision domain. Usually full duplex, no longer need CSMA/CD.

4.3.2 Ethernet Frame Structure

```
| Preamble | Dest MAC | Src MAC | Type | Data | CRC |
```

- **Preamble:** Physical layer synchronization (not counted as frame content)
- **Destination MAC (6 bytes):** Which NIC should receive this frame
- **Source MAC (6 bytes):** Who sent this frame
- **Type:** Which upper-layer protocol does the data belong to
- **Data/Payload:** 46 ~ 1500 bytes, usually contains IP datagrams
- **CRC:** Error detection. If mismatch, discard frame directly. Does not handle error correction or retransmission.

4.4 Link Layer Switches

- **Function:** Store and forward frames; Layer 2 devices.
- **Transparency:** Hosts are unaware of the switch's existence.
- **Self-learning:** Build forwarding tables by observing source MAC addresses. If destination is known, forward selectively; if unknown, flood.
- **Comparison with routers:** Routers handle IP (L3), switches handle MAC (L2). Switches are plug-and-play, routers need configuration.

4.5 Virtual LANs (VLANs)

- **Problem without VLANs:** The entire LAN is a broadcast domain, all Layer 2 broadcasts must flood throughout the entire LAN, causing performance degradation.
- **Purpose:** Logically divide broadcast domains on physical switches, isolate traffic, improve security.
- **Implementation:** Port-based division. Cross-switch uses **Trunk ports** and **802.1Q** protocol (insert Tag in frame to mark VLAN ID).

Comprehensive Case: A Day in the Life of a Web Request

Protocol steps from turning on PC to getting Web page:

- **Ethernet link establishment:** PC powers on, NIC initializes
- **DHCP:** Obtain IP, mask, gateway IP, DNS IP (Discover → Offer → Request → ACK)
- **ARP:** Obtain gateway router MAC
- **DNS:** Resolve domain name to Web server IP
- **TCP:** Three-way handshake, establish connection
- **HTTP:** Request web page, receive HTML content

MPLS (Multi-Protocol Label Switching)

- **Position:** "Layer 2.5" protocol, operates between Layer 2 and Layer 3
- **Principle:** High-speed forwarding based on fixed-length labels, rather than IP longest prefix matching.
- **Advantages:** Traffic Engineering, VPN Support, Fast Reroute.
- **Key terms:** LSR (Label Switching Router), LER (Label Edge Router), LDP (Label Distribution Protocol), FEC (Forwarding Equivalence Class), LSP (Label Switched Path).
- **Operations:** Label push, swap, pop.

Data Center Networks

- **Challenges:** High-density hosts, massive traffic, load balancing.
- **Network Architecture:** Multi-tier (TOR → Tier-2 → Tier-1 → Border router). Multipath (Fat-tree architecture) to increase redundancy and throughput.
- **New Technologies:** RDMA (RoCE), SDN, ECN (congestion control).

VPN & Enterprise WANs

- **VPN (Virtual Private Network):** Establish encrypted tunnels on public networks to transmit private data.
- **Classification:** Site-to-Site (IPsec, GRE, MPLS VPN) and Remote Access (SSL VPN, IPsec).
- **Private Networks:** Use RFC1918 private IP addresses (10.x.x.x, 192.168.x.x, etc.).
- **MPLS VPN:** VPN services provided by operators, isolating different customers' traffic through labels.